Rapport d'Atelier : Git & GitHub Collaboratif

Sujet : Développement Web en Équipe **Organisation :** Binôme (Développeur 1 & 2) **Durée :** 1h30 - 2h

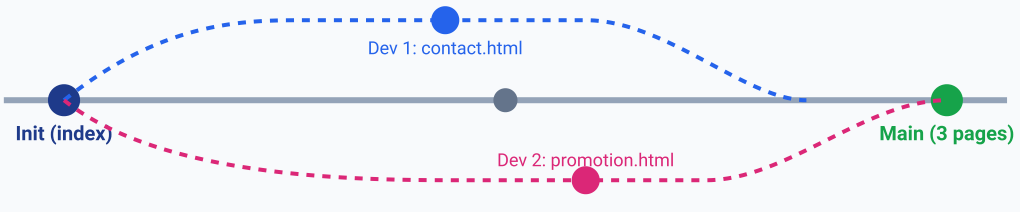
Objectif Global de l'Atelier

Apprendre à collaborer efficacement sur un même projet de site web à l'aide de Git et GitHub sans altérer directement la branche principale (`main`/`master`). Le flux de travail repose sur la création de branches thématiques et l'intégration des modifications via des **Pull Requests**.

1. Synthèse de la Structure & Répartition des Rôles

Étape	Intervenant	Branche	Action / Fichier Produit	Commande / Outil Principal
Init & Base	Développeur 1	`main` / `master`	Création du dépôt distant + `index.html`	`git clone` / `git push`
Fonctionnalité 1	Développeur 1	`contact`	Création de la page `contact.html`	`git checkout -b contact`
Invitation	Développeur 1	-	Ajout du Dev 2 dans Settings GitHub	GitHub Settings > Collaborators
Fonctionnalité 2	Développeur 2	`promotion`	Création de la page `promotion.html`	`git checkout -b promotion`
Revue & Fusion	Binôme	`main` ← Branches	Ouverture, revue et fusion des PRs	GitHub Pull Requests (Merge)
Synchronisation	Binôme	`main` (local)	Rapatriement des 3 pages sur le local	`git pull origin main`

2. Architecture du Workflow Git Collaboratif



3. Réponses aux Questions de Synthèse & Concepts Clés

1. Quelle est la différence entre un dépôt local et un dépôt distant ?
Le dépôt local réside sur la machine du développeur (permet les commits offline). Le dépôt distant est hébergé sur un serveur (ex: GitHub) et sert de point central pour partager le code et collaborer.

2. Pourquoi chaque développeur travaille-t-il sur une branche distincte ?
Cela isole les nouvelles fonctionnalités en cours de développement, évite d'impacter la version stable sur la branche principale et prévient les conflits directs lors des modifications simultanées.

3. Quel est le rôle de `origin` dans `git push origin contact` ?
`origin` est l'alias par défaut qui pointe vers l'URL du dépôt distant sur GitHub. La commande indique d'envoyer les commits de la branche locale `contact` vers ce dépôt distant.

4. Pourquoi utilise-t-on `--set-upstream` lors du premier push ?
Il crée un lien de suivi ("tracking") entre la branche locale et la branche distante. Cela permet de simplifier les commandes futures en utilisant simplement `git push` ou `git pull`.

5. Quelle est la différence entre `git push` et une Pull Request ?
`git push` transfert le code vers le serveur GitHub. Une Pull Request est une demande formelle sur l'interface GitHub pour réviser, discuter et valider l'intégration de ce code dans la branche principale.

6. Une Pull Request fusionne-t-elle automatiquement la branche ?
Non. La PR est une phase de revue. La fusion (Merge) n'intervient qu'après validation explicite par un membre de l'équipe ou le responsable du projet.

7. Pourquoi faut-il récupérer la branche principale après une fusion ?
Parce que la fusion s'effectue sur GitHub (distant). Exécuter `git pull origin main` sur sa machine est indispensable pour mettre à jour son propre dépôt local avec le travail validé de l'équipe.

8. Bilan des branches créées pendant l'atelier
Trois branches au total : `main`/`master` (branche principale stable), `contact` (développement de la page contact par Dev 1), et `promotion` (développement des promotions par Dev 2).

Conclusion de l'Atelier

L'atelier a permis de valider avec succès le workflow GitHub complet : initialisation, clonage SSH, création de branches isolées, publication vers `origin`, revue de code par Pull Request et synchronisation finale des dépôts locaux avec les trois pages web connectées (`index.html`, `contact.html`, `promotion.html`).